

왕복 번역의 근사 고정점으로 측정한 프로그래밍 패러다임 간 코드 번역 거리

(Abstract) 대규모 언어 모델(Large Language Model, LLM)의 코드 번역에서 패러다임 차이가 번역 안정성에 주는 영향을 왕복 번역(Round-Trip Translation, RTT)의 근사 고정점으로 측정하였다. 편도 번역마다 생성된 서로 다른 (언어, 소스) 상태의 수를 거리 d_n 으로, 모든 생성 코드가 기능 검사를 통과하고 상태 상한 안에서 인접 복원 코드의 유사도가 0.85에 도달한 시도의 비율을 P_{sf} 로 정의하였다. C++, Haskell, Prolog로 검증한 15개 문제에 GLM-5.3 Flash를 적용하여, 사전 등록된 규칙으로 8개 경로 690회 시도를 수행하였다. C++에서 같은 명령형인 Python과 Java를 거친 경로의 P_{sf} 는 0.83와 0.75로, 함수형 Haskell(0.17)과 논리형 Prolog(0.03)를 거친 경로보다 높았고, 네 비교의 Bonferroni 구간은 모두 0을 포함하지 않았다. 성공 시도의 d_n 은 대조 경로가 약 1 작았으나 구간이 0을 포함하였다. 실패의 대부분은 중간 언어의 생성 오류였으며 편도 기능 통과율은 Haskell 73~90%, Prolog 36~46%였다. 결과는 패러다임 가설과 같은 방향이지만, 그 차이가 모델의 언어별 생성 능력과 분리되지 않음을 보인다.

1. 서론

LLM의 코드 번역은 문법 변환을 넘어 자료 구조, 제어 흐름과 입출력 동작의 보존을 요구한다. 번역 품질은 흔히 참조 번역이나 단위 테스트로 평가되지만[3], 패러다임이 다른 언어 사이에서 모델의 번역이 얼마나 안정적인지를 하나의 척도로 비교하기는 어렵다. 자연어에서는 RTT를 고정점에 도달할 때까지 반복하고 그 과정의 고정점 집합(Fixed-Point Set, FPS) 크기를 언어 간 거리로 쓰는 방법이 제안되었다[1,2].

본 연구는 이 방법을 실행 가능한 코드로 옮겨 "패러다임이 다른 언어를 거칠수록 거리가 길어진다"는 가설을 검증한다. 기여는 세 가지다. 첫째, 기능 검사와 결합한 코드 상태 기반 FPS 거리와 정지 기준을 정의한다. 둘째, 반복 시도와 사전 등록된 종료 규칙으로 도달률과 거리를 분리해 보고한다. 셋째, 같은 패러다임의 대조 경로를 두어 관찰된 차이의 원인을 진단한다.

2. 방법

2.1. 거리와 정지 기준

출발 언어 A의 원본을 x_0 , t번째 중간 코드를 y_t , A로 복원한 코드를 x_t 라 하면 경로는 $x_0 \rightarrow y_1 \rightarrow x_1 \rightarrow y_2 \rightarrow x_2 \rightarrow \dots$ 이다. j번째 편도 번역이 만든 언어와 소스를 (L_j, z_j) , 줄바꿈만 LF로 통일한 소스의 SHA-256 해시를 h 라 할 때 상태 집합과 크기는 다음과 같다.

$$S_0 = \{(A, h(x_0))\}, \quad S_j = S_{j-1} \cup \{(L_j, h(z_j))\}, \quad D_j = |S_j|$$

복원 단계에서 인접 복원 코드의 유사도 $\text{Sym}(x_{t-1}, x_t)$ 가 처음 $\theta = 0.85$ 이상이 되면 근사 고정점에 도달한 것으로 보고 $d_n = D_j$ 를 기록한다. Sym 은 JPlag[4] text 모드의 평균 유사도(최소 일치 9토큰)이며 정지 기준으로만 쓴다. 모든 생성 코드는 13개 평가 입력으로 검사하고, 코드 추출·컴파일·실행

실패, 오답, 시간 초과가 나오면 기능 실패로 종료한다. $D_j > 10$ 이면 상한 초과, 편도 20회에 이르면 예산 소진으로 종료한다. 성공 판정은 θ 기준이며, 민감도 분석을 위해 0.90에 도달할 때까지 번역을 이어간다.

경로·문제마다 독립 시도를 5회 수행하고, P_{sf} 는 문제별 성공 비율의 동일 가중 평균으로 집계한다. d_n 은 성공 시도에서만 정의되므로 P_{sf} 와 함께 읽는다. 신뢰구간은 문제를 재표집하는 부트스트랩(2,000회)으로 구하고, 비교 쌍 수로 Bonferroni 보정한다.

2.2. 실험 설정

IPOD 6문제와 LeetCode 9문제에 대해 C++, Haskell, Prolog 기준 코드 45개가 13개 입력을 모두 통과함을 확인하였다. H2는 세 언어 사이의 6개 방향 경로, H1은 C++에서 Python·Java를 거치는 대조 경로와 Haskell·Prolog를 거치는 처리 경로의 비교이다. 모델은 GLM-5.3 Flash(temperature 0.6, top_p 0.95, 추론 강도 low, 시도별 seed)이며 도구는 GCC 14.2, GHC 9.10.3, SWI-Prolog 10.0.2, JPlag 6.3.0이다. 3문제 예비 실험 후 규칙을 동결했고, 총 2,480회 호출에 비용은 약 0.43달러였다. 무효 시도는 없었다.

사전 점검에서 같은 편집을 세 언어 기준 코드에 적용했을 때 공백·주석·연산자·한 줄 삭제에 대한 Sym 변화는 언어 간 0.05 이내였다. 다만 식별자 개명은 짧은 Haskell 코드에서 더 크게 떨어졌고(평균 0.62, C++ 0.79), 연산자 변경에는 거의 반응하지 않았다. 따라서 Sym 은 텍스트 안정성의 조작적 기준이며, 의미 보존은 기능 검사가 담당한다.

3. 결과 및 고찰

표 1은 8개 경로의 주 결과이며, 모든 경로는 계획한 75회 시도를 완료하고 사후 검사를 통과하였다. H1의 판정 규칙은 실험 전에 정하였다. 대조 경로의 P_{sf} 가 높고 d_n 이 작으며 문제 단위

구간이 0을 포함하지 않으면 지지, 두 지표 중 하나만 만족하면 부분 지지, 방향이 반대이거나 구간이 0을 포함하면 기각으로 본다.

3.1. H1: 같은 패러다임 대조 경로와의 비교

대조 경로의 P_{sf} 는 처리 경로보다 0.57~0.80 높았고, 네 비교의 Bonferroni 95% 구간 하한은 모두 0.36 이상이었다. 임계값을 0.80이나 0.90으로 바꾸거나 예비 실험 문제를 빼도 결론은 같았다. 성공 시도의 d_n 차이는 -1.1~0.2로 대조 경로가 작은 방향이었으나, 두 경로가 함께 성공한 문제가 2~6개뿐이라 구간이 0을 포함하였다. 사전 규칙에 따라 H1은 P_{sf} 에서 지지, d_n 에서 방향만 일치하는 부분 지지로 판정한다.

표 1. 경로별 도달률과 성공 조건부 거리 ($\theta = 0.85$, 경로당 75회)

경로	성공	P_{sf} [95% CI]	d_n 평균 (n)	기능 실패	상한
C→Py→C	62/75	0.83 [0.67, 0.95]	5.7 (62)	1	12
C→J→C	56/75	0.75 [0.57, 0.91]	5.4 (56)	8	11
C→H→C	13/75	0.17 [0.07, 0.31]	6.4 (13)	56	6
C→P→C	2/75	0.03 [0.00, 0.07]	6.0 (2)	71	2
H→C→H	4/75	0.05 [0.00, 0.15]	6.8 (4)	53	18
H→P→H	0/75	0.00 [0.00, 0.00]	NA	72	3
P→C→P	0/75	0.00 [0.00, 0.00]	NA	73	2
P→H→P	0/75	0.00 [0.00, 0.00]	NA	71	4

C=C++, H=Haskell, P=Prolog, Py=Python, J=Java. 위 네 경로가 H1, 아래 네 경로와 C→H→C, C→P→C가 H2이다. 예산 소진과 무효는 모든 경로에서 0이다.

3.2. H2: 세 패러다임 사이의 방향 경로

C→H→C가 0.17로 가장 높았고 H→C→H 0.05, C→P→C 0.03 순이었으며, Prolog가 포함된 나머지 세 경로는 성공이 없었다. C++와 Haskell의 방향 차이는 0.12 [-0.05, 0.29]로 비대칭을 확인할 수 없었고, 경로 순위는 세 임계값에서 같았다.

3.3. 차이는 어디에서 왔는가

편도 번역의 기능 통과율은 생성 언어에 따라 C++ 98~100%, Python 100%, Java 96%, Haskell 73~90%, Prolog 36~46%였다. 성공에는 중간 언어 생성이 여러 번 연속으로 통과해야 하므로, 처리 경로의 낮은 P_{sf} 는 대부분 이 생성 실패에서 나온다. 기능 실패가 없던 시도만 보면 도달 비율은 Python 62/74, Java 56/67, Haskell 13/19, Prolog 2/4로 차이가 줄었다.

그럼에도 기능이 보존된 복원 코드끼리의 인접 Sym 중앙값은 Python 0.64, Java 0.62, Haskell 0.24, Prolog 0.16로(Fig. 1 아래), 다른 패러다임을 거친 C++는 왕복마다 더 크게 다시 쓰였다. 또한 H2의 코드 추출 실패 124건은 모두 한 응답에 초안과 수정본 등 여러 코드 블록을 낸 경우였고 106건이 Prolog 생성이었다. 사후 진단으로 마지막 블록만 평가하면 65건이 통과했으나, 이는 주 결과에 반영하지 않았다.

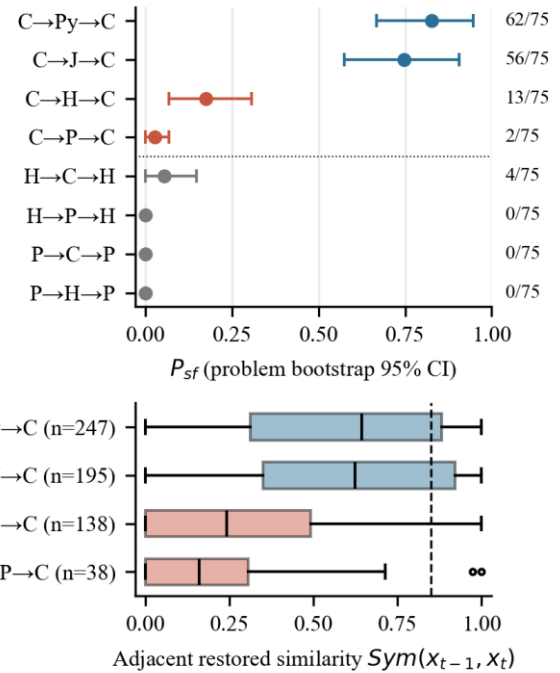


Fig. 1. 경로별 P_{sf} 와 문제 단위 부트스트랩 95% 구간(위, 점선 위는 H1 네 경로), H1 경로의 인접 복원 유사도 분포(아래, 점선은 $\theta = 0.85$). 파란색은 같은 패러다임 대조, 빨간색은 처리 경로이다.

4. 결론

코드 RTT에서 도달률 P_{sf} 와 성공 조건부 거리 d_n 을 분리하여 측정하고, 같은 패러다임 대조 경로와 비교하였다. 이 모델과 설정에서 C++는 Python·Java를 거칠 때 근사 고정점에 훨씬 자주 도달하였고, 이는 패러다임이 다를수록 거리가 길어진다는 가설과 같은 방향이다. 그러나 주된 기제는 중간 언어의 생성 실패였으며, Python·Java는 명령형이면서 학습 자료가 가장 많은 언어이기도 하다. 따라서 결과를 패러다임 자체의 거리로 해석하지 않고, 모델의 언어별 능력과 결합된 관찰로 한정한다. d_n 은 공통 성공 문제가 적어 비교력이 낮았고, JPlag text 유사도는 짧은 코드와 연산자 변경에 한계가 있다. 후속 연구로 상위 모델에서의 재실행, 기능 실패와 수렴을 분리한 조건부 지표의 사전 등록, 문제 수 확대를 수행할 계획이다.

참고문헌

- [1] Cho, C. et al. Computing the Distance Between Languages Based on Round Trip Translation and Fixed Points. *Proc. Korea Software Congress* 2025, 1467–1469 (2025)
- [2] Crone, N. et al. Quality Estimation Using Round-Trip Translation with Sentence Embeddings. *arXiv*: 2111.00554 (2021)
- [3] Roziere, B. et al. Unsupervised Translation of Programming Languages. *Adv. Neural Inf. Process. Syst.* 33, 20601–20611 (2020)
- [4] Prechelt, L. et al. Finding Plagiarisms among a Set of Programs with JPlag. *J. Univers. Comput. Sci.* 8, 1016–1038 (2002)